University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Localization of a Mobile Robot

Srikanth Nanda
First examiner: Prof. Dr. Elmar Wings
Second examiner: Prof. Dr. Jonas Schwarz

21. September 2026

Table of Contents I

1 Introduction
2 Problem Description
3 Challanges
4 Domain Hardware
5 Proposed Solution
6 Methodology
7 Development
8 Evaluation
9 Monitoring
10 Deployment
11 Results
12 Conclusion
13 Next Steps
14 Delighter

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Introduction

Introduction I

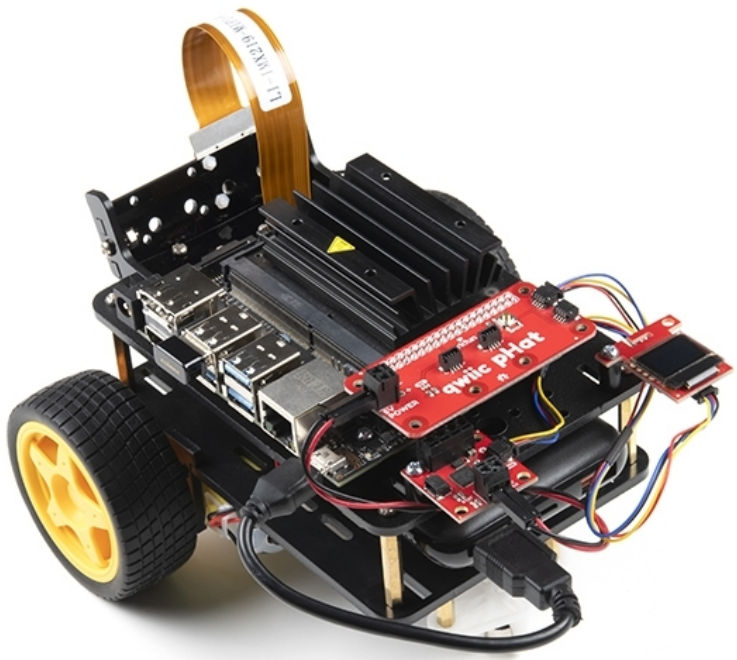


Abbildung: Localization in a Mobile Robot

Introduction II

- Localization lets a robot estimate its **position and orientation** inside an environment
- Outdoor systems rely on **GPS**; indoors the signal is attenuated, reflected (multipath) and interfered with, so satellite positioning is not usable.
- Indoor alternatives: RF-based (Wi-Fi, Bluetooth), ultra-wideband, inertial measurement, and **vision-based** techniques.
- Vision-based localization estimates pose directly from image data; **monocular SLAM** additionally builds a map of the environment while tracking motion.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Problem Description

Problem Description I

- **Task:** Indoor localization on a mobile robot.
- Identify the mobile robot hardware and establish indoor localizaition.
- Motion must therefore be recovered by **precise feature matching and tracking** across consecutive frames.

Challanges

Challanges I

1. Geometric and perceptual challenges
   - Absence of depth information ⇒ **scale ambiguity**; absolute distances cannot be estimated from monocular views.
   - Feature-based methods depend on **distinguishable texture**, which is not present in all indoor scenes (plain walls, reflective floors).
   - **Temporal drift** accumulates over long trajectories without loop closure.
2. Environmental challenges
   - Illumination changes, dynamic objects, and occlusions degrade tracking.
   - Indoor environments introduce significant uncertainty affecting localization accuracy.

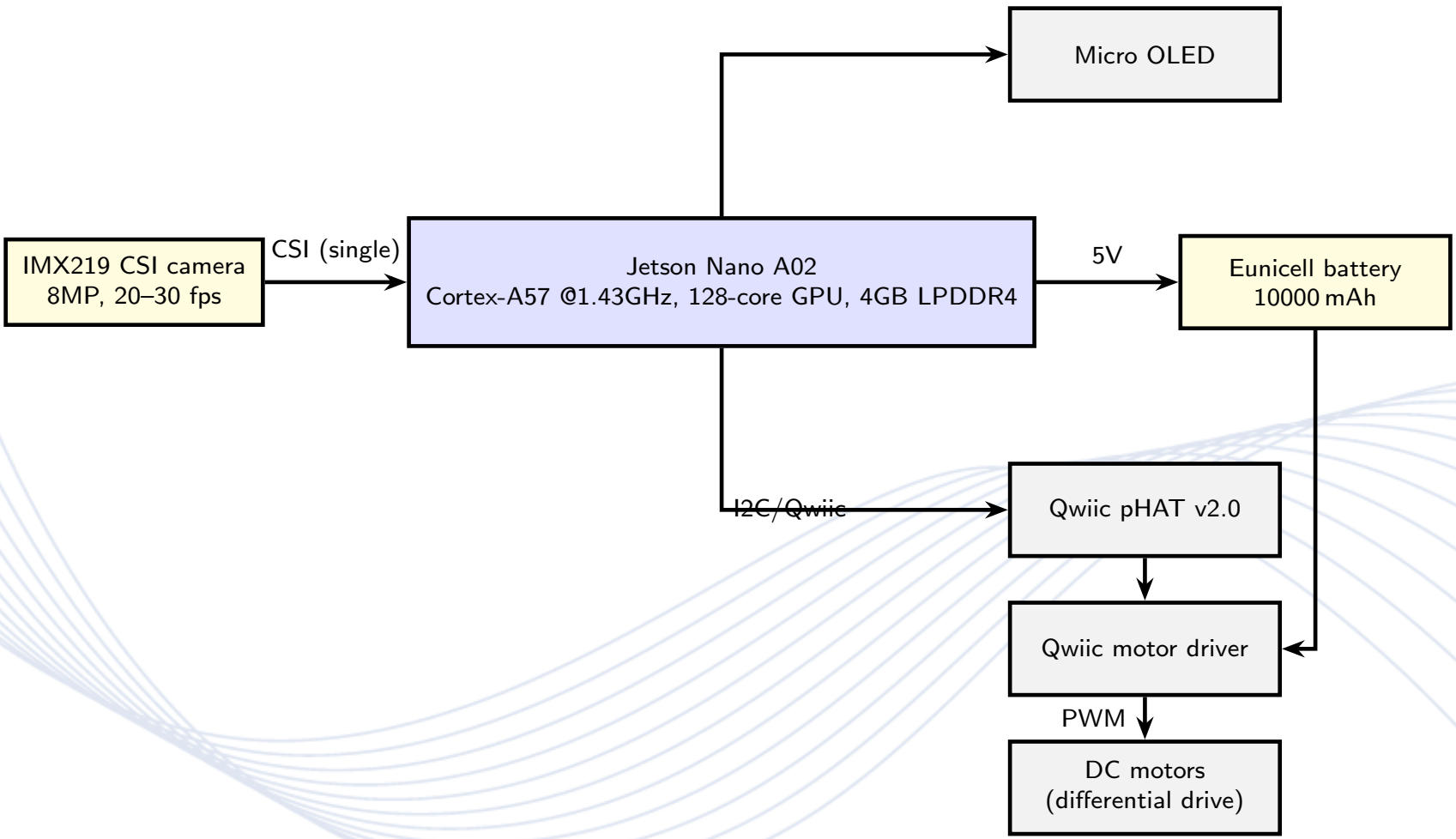
Challanges – Embedded Platform I

- **Compute budget:** the Jetson Nano's shared 4 GB memory and modest GPU limit the algorithmic complexity that can run in real time – heavier stereo or LiDAR pipelines are infeasible.
- **Real-time safety:** obstacle reaction must happen within the control loop period, so a heavy semantic-segmentation or object-detection network is not an option.
- **Power:** battery-powered operation makes latency and energy per inference first-class design criteria.
- **Legacy software ecosystem:** JetPack 4.3 pins Ubuntu 18.04 / ROS Melodic and rosbuild-era tooling, which turned out to be the dominant practical obstacle during integration.
- **Dual use of one sensor:** the same camera stream must simultaneously feed the SLAM front-end and the collision classifier.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Domain Hardware

Hardware Platform I



Hardware Description I

- **SparkFun JetBot v2.1** – differential-drive chassis,Sparkfun Qwiic pHAT v2.0, Qwiic motor driver, micro OLED, Eunicell 10000 mAh battery pack.
- **NVIDIA Jetson Nano Developer Kit A02**
  - quad-core ARM Cortex-A57 @ 1.43 GHz,
  - 128-core Maxwell GPU, ≈472 GFLOPS,
  - 4 GB 64-bit LPDDR4, 25.6 GB/s bandwidth,
  - 5 W / 10 W power modes at 5 V.
- **Leopard Imaging IMX219** 8 MP H-136 CSI camera, front-mounted, 20–30 fps.
- **Decisive constraint:** the A02 carrier exposes a **single usable CSI interface** monocular vSLAM as the main geometrically grounded option.
- **Software baseline:** JetPack 4.3, Ubuntu 18.04, ROS Melodic.

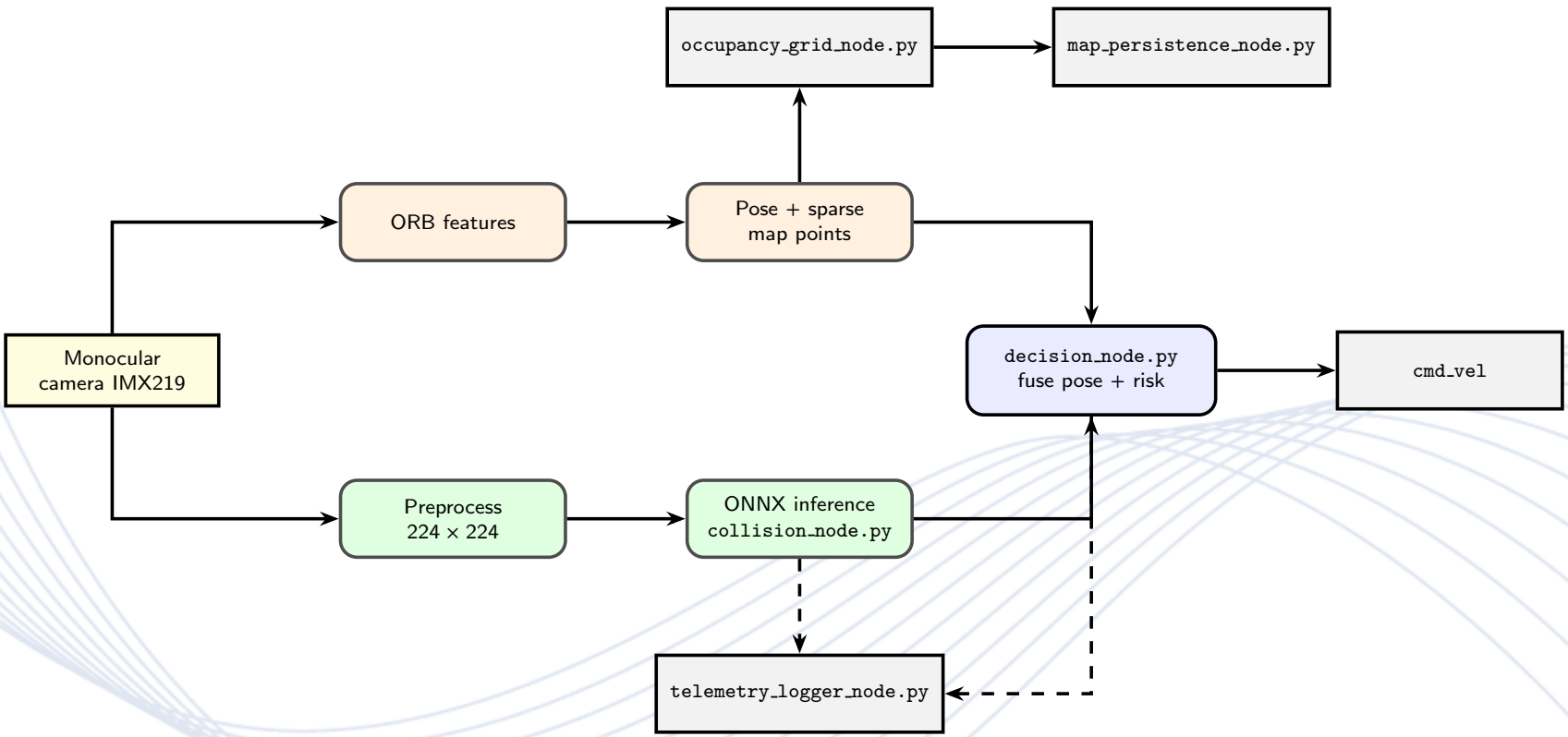
University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Proposed Solution

Proposed Solution I

1. Addressing scale ambiguity and texture dependence
   - **Multi-view geometry and bundle adjustment** – depth inferred by triangulating feature correspondences across multiple frames.
   - **Loop closure** corrects accumulated drift and restores global map consistency.
   - **ORB descriptors** give invariance to rotation, scale and illumination, improving matching in low-texture scenes.
2. Addressing environmental and computational limits
   - **Place recognition and relocalization** to recover from tracking loss.
   - **Outlier rejection** (RANSAC, robust loss) and temporal smoothing against dynamic elements.
   - **Keyframe-based SLAM with local map optimization** to keep real-time performance on the Nano.

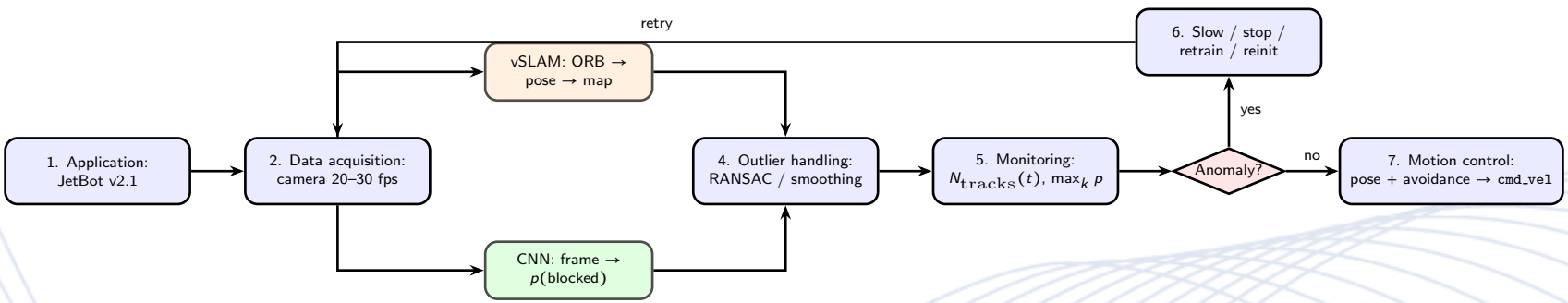
Flowchart System Architecture I



System Architecture I

- Single camera stream is **shared** by two parallel branches:
  - **vSLAM branch** – ORB features → pose + sparse map points,
  - **CNN branch** – free / blocked classification → collision probability.
- Modular **ROS nodes**, each with explicit topics and parameters:
  - `collision_node.py` – ONNX inference, publishes collision probability,
  - `decision_node.py` – fuses pose + risk, publishes `cmd_vel`,
  - `occupancy_grid_node.py` – grid from pose and map points,
  - `map_persistence_node.py` – save / reload point-cloud snapshots,
  - `telemetry_logger_node.py` – CSV logging at 1 Hz.
- `run_jetbot_all.sh` is a configurable orchestration layer with feature toggles (encoder odometry, telemetry, map persistence, CNN collision, camera publishing).

Flowchart of Pipeline I



Hybrid Pipeline I

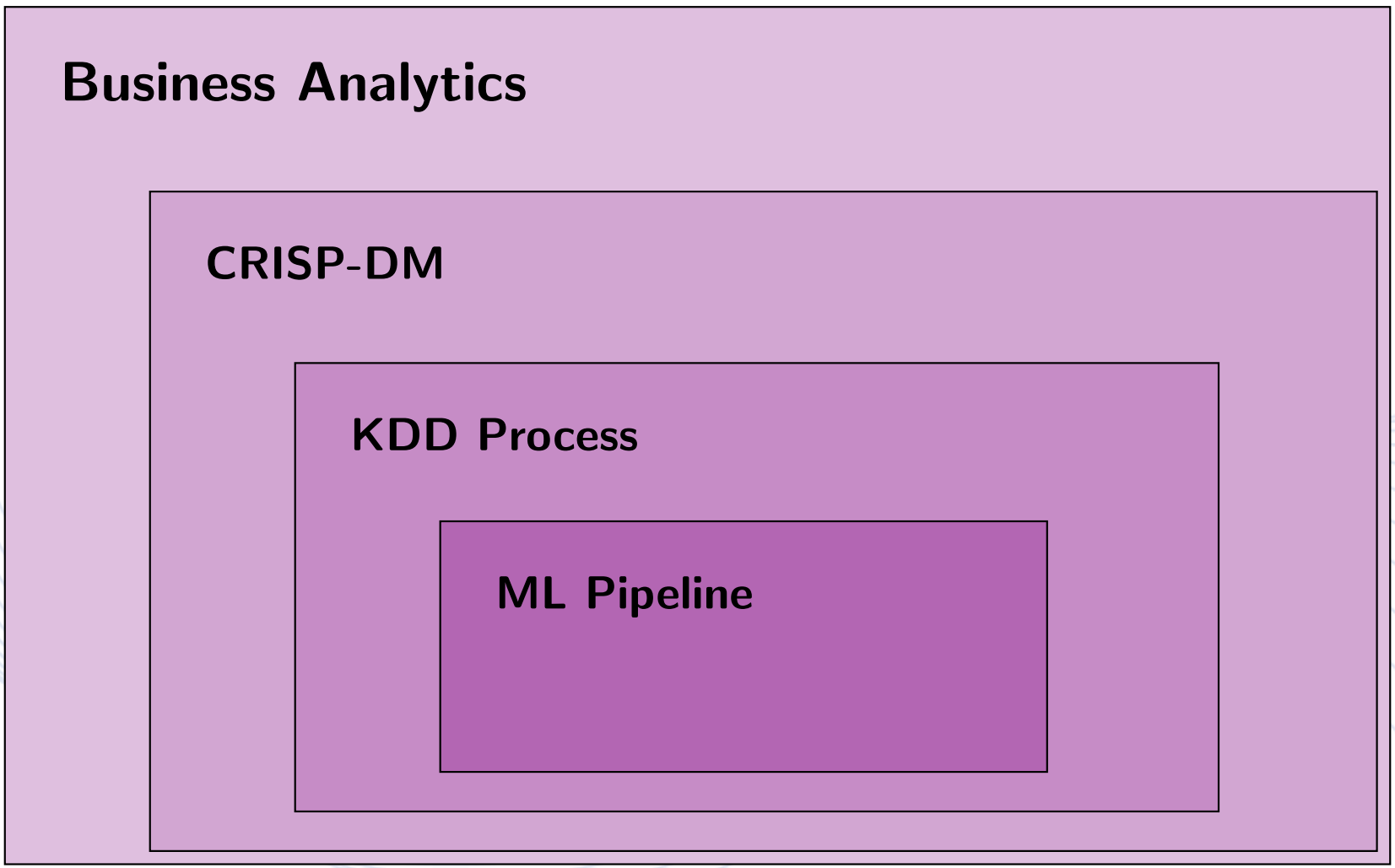
1. **Application** – indoor robot navigation (JetBot v2.1).
2. **Data acquisition** – front-facing monocular camera at 20–30 fps.
3. **Parallel perception**
   - vSLAM: ORB features → pose → sparse map,
   - CNN: preprocessed frame → $p(\text{blocked})$.
4. **Outlier handling** – RANSAC / robust loss on the geometric side, temporal smoothing and safety rules on the learned side.
5. **Monitoring** – number of tracked features $N_{\text{tracks}}(t)$ and maximum class probability $\max_k p(y = k \mid I_t)$.
6. **Anomaly response** – reduce speed, safe stop, retrain; persistent anomalies trigger reinitialization and new data collection.
7. **Motion control** – pose + avoidance decision → bounded wheel command.

Methodology

Methodology I

- No single framework covers the work; **three complementary layers** are combined:
  - **CRISP-DM** structures the project at engineering level,
  - the **KDD process** governs how raw sensor data becomes usable knowledge,
  - a standard **ML pipeline** covers the concrete CNN implementation.
- **Justification**
  - the single CSI interface forces monocular vSLAM,
  - real-time safety forces a lightweight learned classifier,
  - the ROS design allows every node to be validated independently before integrated testing.
- An ML system is more than its algorithm: it also comprises the data pipeline, the evaluation protocol, and the deployment and monitoring infrastructure.

CRISP-DM I



CRISP-DM II

Task	Navigation defined as monocular vSLAM plus CNN-based collision avoidance on embedded hardware largely forced by the single CSI camera interface.
Database	Camera stream, pose topic and collision-probability topic characterized by quantity, quality and known failure modes (illumination sensitivity, low texture, scale ambiguity).
Data Preparation	Captured frames cleaned, deduplicated and augmented.
Modeling	CNN collision classifier trained and integrated with ORB-SLAM2 pose output.
Evaluation	Classifier and system-level performance quantified against held-out data and integrated runs.
Deployment	Trained model exported to ONNX and promoted into the runtime ROS graph.

KDD I

- **Selection** – forward-facing navigation scenes from the JetBot camera logs: free-path motion, close obstacle approaches, reflective surfaces, low-texture regions.
- **Preprocessing / Cleaning** – `cnn_kdd_transform.py` computes perceptual hashes to catch duplicate frames and a Laplacian-variance blur score to flag low-quality images.
- **Transformation** – resize to 224×224, channel reordering, normalization with fixed channel mean and standard deviation.
- **Data Mining** – `cnn_train_tf.py` trains a compact convnet from scratch with aggressive augmentation and close validation-loss monitoring.
- **Interpretation / Evaluation** – outputs read as collision probabilities, checked offline and in integrated hardware trials.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Development

Development – Datasets I

Two datasets with distinct roles, instead of a conventional DBMS:
Blocked / free image dataset

- Collected directly from the JetBot's onboard monocular camera by teleoperation.
- **Free** – clear, navigable path, no immediate obstacle.
- **Blocked** – wall, furniture or other object within collision range.
- Purpose: train a lightweight CNN able to run in real time on the Nano and modulate motor commands.

Development – CNN Observations I

Critical detail: the identical preprocessing function is reused unmodified at inference time, and the ONNX manifest records input size, channel ordering and normalization so **training–serving skew** can be verified rather than assumed.
Filter counts double with depth: shallow layers capture edges and texture, deeper layers task-specific patterns. Optimised lightweight CNNs on edge devices report up to **65% lower latency** and **45% lower energy** than uncompressed CNNs while staying above 90% accuracy.

Development – SLAM Integration I

- **Dependency compilation:** Pangolin, DBoW2 and OpenBLAS built from source on the Nano.
- **ORB-SLAM2** built successfully as a **standalone library**; repository structure intact.
- **ROS bridge:** catkin workspace with camera integration, containerized bring-up, network configuration.
- **ORB-SLAM3** configured for monocular mode (camera intrinsics, vocabulary path, sensor mode) and validated on `freiburg1_xyz` – tracking, local mapping and loop-closing threads visible in the Pangolin viewer.

Development – Integration blockers I

Integration blockers encountered:

- Python bindings fail with an **undefined symbol** in the Boost/OpenCV converter layer,
- rosbuild-based examples cannot obtain compile flags because `rospack` fails on a `python-catkin-pkg` dependency under JetPack 4.3 / ROS Melodic,
- the launch configuration references a non-existent `orb_slam2_ros` package.

Development – Model Export I

```
2026-07-15 13:20:20.676913: I tensorflow/core/grappler/devices.cc:55] Number of eligible GPUs
(core count >= 8, compute capability >= 0.0): 0
2026-07-15 13:20:20.677097: I tensorflow/core/grappler/clusters/single_machine.cc:356] Startin
g new session
2026-07-15 13:20:20.678032: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:950] ARM64
does not support NUMA - returning NUMA node zero
2026-07-15 13:20:20.678151: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1639] Found dev
ice 0 with properties:
name: NVIDIA Tegra X1 major: 5 minor: 3 memoryClockRate(GHz): 0.9216
pciBusID: 0000:00:00.0
2026-07-15 13:20:20.678223: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcudart.so.10.0
2026-07-15 13:20:20.678285: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcublas.so.10.0
2026-07-15 13:20:20.678333: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcufft.so.10.0
2026-07-15 13:20:20.678374: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcurand.so.10.0
2026-07-15 13:20:20.678422: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcusolver.so.10.0
2026-07-15 13:20:20.678462: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcusparse.so.10.0
2026-07-15 13:20:20.678501: I tensorflow/stream_executor/platform/default/dso_loader.cc:44] Su
ccessfully opened dynamic library libcudnn.so.7
2026-07-15 13:20:20.678624: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:950] ARM64
does not support NUMA - returning NUMA node zero
2026-07-15 13:20:20.678790: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:950] ARM64
does not support NUMA - returning NUMA node zero
2026-07-15 13:20:20.678869: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1767] Adding vi
sible gpu devices: 0
2026-07-15 13:20:20.678935: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1180] Device in
terconnect StreamExecutor with strength 1 edge matrix:
2026-07-15 13:20:20.678968: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1186]      0
2026-07-15 13:20:20.678994: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1199] 0:   N
2026-07-15 13:20:20.679132: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:950] ARM64
does not support NUMA - returning NUMA node zero
2026-07-15 13:20:20.679310: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:950] ARM64
does not support NUMA - returning NUMA node zero
2026-07-15 13:20:20.679489: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1325] Created T
ensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 508 MB memory) -> physical
 GPU (device: 0, name: NVIDIA Tegra X1, pci bus id: 0000:00:00.0, compute capability: 5.3)
2026-07-15 13:20:20.727656: E tensorflow/core/grappler/grappler_item_builder.cc:656] Init node
 conv2d/kernel/Assign doesn't exist in graph
Exported TFLite model successfully
jetbot@jetson-4-3:~/catkin_ws/src/CollisionAvoidanceusingLocalization/scripts$ ls -la models/c
ollision.tflite
-rw-rw-r-- 1 jetbot jetbot 28112 Jul 15 13:20 models/collision.tflite
```

Abbildung: Collision Tensorflow Lite Model Verification

Development – Model Export II

- **Training pipeline:** load and preprocess images, split train/validation/test, build the sequential model, compile with Adam and binary cross-entropy, train with augmentation (flips, brightness jitter).
- **Export chain:** frozen graph → SavedModel → TFLite → **ONNX**, with GPU initialization and TensorFlow pipeline debugging documented at each step.
- **Manifest:** `cnn_dev_to_deploy.py` writes input tensor size, channel ordering and normalization parameters alongside the artifact.
- **Promotion:** `cnn_deploy_model.py` uses hashing and manifest verification – a champion/challenger discipline at small-system scale; a new ONNX artifact is only promoted after its manifest verifies.
- **Data version control:** environment setup and dataset versioning so a retrained model follows the same validated path as the original.

Evaluation

Evaluation I

Can the JetBot maintain a stable pose estimate while avoiding obstacles in real time, i.e. localize, classify risk and issue safe velocity commands without human intervention?

- **Metrics** computed by `cnn_evaluate.py` against a labeled test set: confusion matrix, accuracy, precision, recall, F1, and **average inference latency**; results optionally written to JSON.
- Decision threshold default 0.60; input 224×224, ImageNet-style mean/std normalization, ONNX executed through `cv2.dnn`.
- **Application:** corridor runs, obstacle approaches and recovery scenarios; every run records camera stream, pose, collision probability and command output for post-hoc inspection.

Evaluation – Validation I

Current status is **partial functional validation**, not full system validation.

1. **CNN-only navigation validation** – run the deployed ONNX model, route the decision node to the CNN collision topic, and measure whether the JetBot sustains obstacle-free reactive motion in a known corridor.
2. **Map-collision and occupancy-grid validation** – exercise the occupancy-grid and map-persistence nodes with recorded or simulated pose and sparse point-cloud inputs, no live SLAM required.
3. **Split-environment validation** – ONNX and control stack on the JetBot, ORB-SLAM2/3 trajectory and map generation in a better-supported offline environment, then compare expected SLAM-enhanced behaviour against deployable JetBot behaviour.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Monitoring

Monitoring I

Idea: model and data quality degrade *quietly* once a system leaves development; ML systems fail silently because predictions keep arriving without any crash. The same telemetry used to validate lab experiments must also catch degraded camera input, stale pose estimates or abnormal collision probabilities in autonomous operation at near-zero compute overhead.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Deployment

Deployment I

- **Pipeline:** camera data collection on the JetBot → offline training of the collision-risk classifier → copying the exported model and ROS package onto the NVIDIA Jetson Nano.

```
jetbot@jetson-4-3:~/catkin_ws/src/CollisionAvoidanceusingLocalization/scripts$ python3 cnn_deploy_model.py --model models/collision.onnx
Traceback (most recent call last):
  File "cnn_deploy_model.py", line 105, in <module>
    main()
  File "cnn_deploy_model.py", line 96, in main
    args.swap_rb,
  File "cnn_deploy_model.py", line 48, in deploy_model
    shutil.copy2(model_path, dest_path)
  File "/usr/lib/python3.6/shutil.py", line 263, in copy2
    copyfile(src, dst, follow_symlinks=follow_symlinks)
  File "/usr/lib/python3.6/shutil.py", line 104, in copyfile
    raise SameFileError("{!r} and {!r} are the same file".format(src, dst))
shutil.SameFileError: 'models/collision.onnx' and 'models/collision.onnx' are the same file
jetbot@jetson-4-3:~/catkin_ws/src/CollisionAvoidanceusingLocalization/scripts$ ^C
jetbot@jetson-4-3:~/catkin_ws/src/CollisionAvoidanceusingLocalization/scripts$ python3 cnn_deploy_model.py \
>   --model models/collision.onnx \
>   --dest deployed/collision.onnx \
>   --manifest deployed/collision_manifest.json
Deployed model to deployed/collision.onnx
Manifest: deployed/collision_manifest.json
SHA256: ec47b561f51dca801786525e87701251436d5b1b2b6cb706247370d448f22c24
```

Abbildung: Successful Deployment of CNN Model

Deployment Phase 2 I

- Pose estimator, collision inference node and motion decision node all live in **one ROS workspace**, keeping the control loop local to the robot.
- Reproducible versioning ensures the exact artifact validated offline is the one that runs on the robot.
- `run_jetbot_all.sh` supports copying a chosen model into `models/collision.onnx` and can route the control node either to map-based collision output or to a CNN-based collision topic.

Deployment – Orchestration I

Orchestration of `run_jetbot_all.sh`:

1. Parse command-line arguments and set feature flags.
2. **Checks-only mode?** → run requested pre-flight checks: `--camera-test`, `--calibrate`, `--qwiic-test`.
3. Otherwise: deploy the CNN model if `--model-path` is given.
4. Start the camera publisher if `--start-camera-pub`.
5. Launch `system.launch` with all feature toggles and the selected collision topic – camera driver, SLAM node, occupancy grid, collision node, decision controller and telemetry logger, in order and with startup delays.
6. Optionally run `cnn_pipeline_monitor.py` (`--monitor`) and `cnn_data_collect.py` (`--collect-data`).
7. Wait for `roslaunch`, then clean up.

Results

Results – Perception and Localization I

- **CNN training completed:** final-epoch logs confirm convergence and a saved model checkpoint; precision, recall and accuracy computed on the validation set.
- **ONNX evaluation:** accuracy and average latency of the exported ONNX model compared against the original TensorFlow model verified after training.
- **Live test:** collision-detection model exercised on live JetBot camera input.
- **ORB-SLAM3 on TUM RGB-D:** successful execution on `freiburg1_xyz`: real-time sparse 3D point cloud, keyframe trajectory and live feature tracking in Pangolin; full camera trajectory generated and saved for accuracy evaluation.
- **Build validation:** ORB-SLAM3 shared libraries linked with no unresolved symbols; ROS2/colcon workspace configured for publishing localization data.

Results – Integration Findings I

1. Observations
   - ORB-SLAM2 built as a standalone library; complete Python navigation stack with deployment script, telemetry logger, occupancy-grid node and reactive decision node.
   - The decision node implements a usable reactive policy: slow-down and stop thresholds, recovery turning, optional reversing, heading stabilization.
2. Drawbacks
   - **ROS integration** blocked – rosbuild examples cannot obtain compile flags after `rospack` fails on `python-catkin-pkg` under JetPack 4.3 / ROS Melodic.
   - Bring-up reaches launch but shuts down on the missing `orb_slam2_ros` package: only `/rosout` remains, i.e. **no navigation nodes active**.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Conclusion

Conclusion I

- The work produced a **well-structured onboard navigation software stack**, but not yet a stable end-to-end deployment of monocular ORB-SLAM2 on the Jetson Nano under JetPack 4.3 / ROS Melodic.
- Strongest practical outcome: a **deployable ONNX-based collision-avoidance pipeline** with ROS orchestration, telemetry logging and modular support for occupancy grids and map persistence.
- The central limitation is not architectural design but the **fragility of the software integration boundary** between ORB-SLAM2, ROS and Python bindings on this platform.

Self-critical I

- Effort went into high-risk integration before a minimal working baseline was frozen; SLAM should have been an incremental extension, not an early dependency.
- Device-level trajectory results could not be reported for confidentiality reasons, so the **TUM RGB-D benchmark** serves as a controlled substitute, it isolates algorithmic performance but not the real corridor's lighting, texture and motion characteristics.
- **Open questions:** whether the JetPack 4.3 limitation is repairable at acceptable cost; how robust CNN-only navigation is across lighting and texture variation; and whether persistent map reuse can support long-term autonomy.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Next Steps

Next Steps I

- **Minimal successful run:** refactor the launch configuration so all `orb_slam2_ros` references become optional flags, start the stack in CNN-only mode, and verify live topics for camera input, collision output and non-zero `cmd_vel` in a short bench test.
- **Repeatable indoor trials:** run the ONNX collision pipeline on a fixed test route with CSV telemetry enabled, reporting run duration, command stability, collision events and battery state instead of only software architecture.
- **Platform upgrade:** flashing JetPack 4.5 is performed using BalenaEtcher in compatible SD Card.

Next Steps II

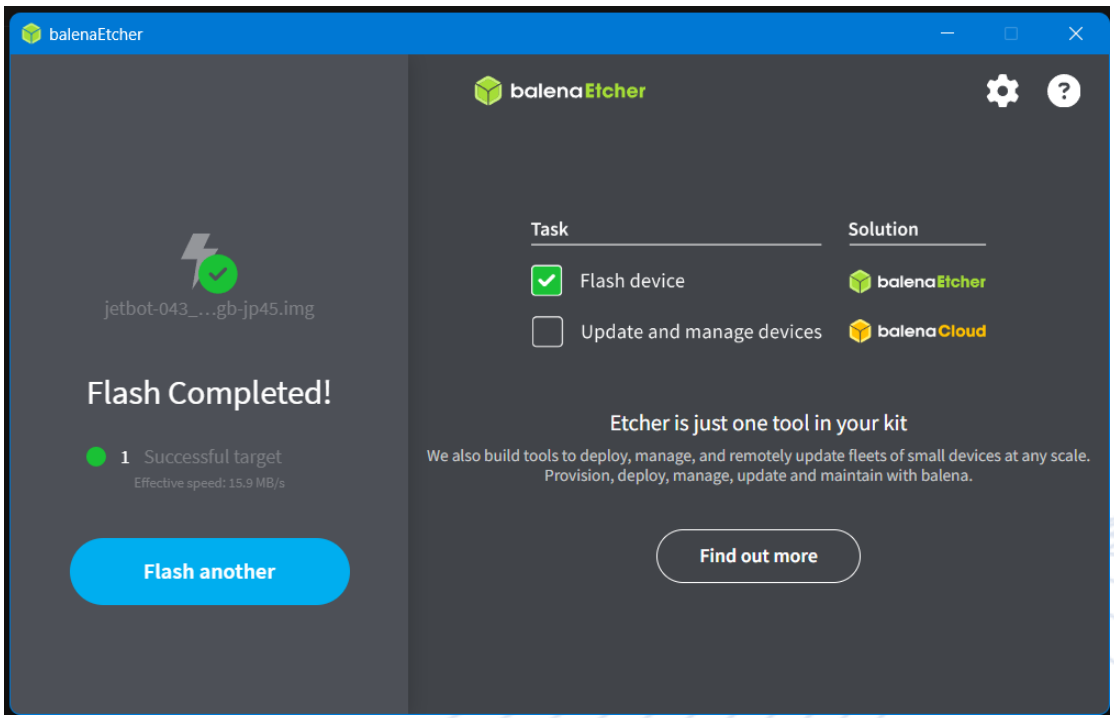


Abbildung: Flashing Successful NVIDIA Jetbot - Jetpack 4.5 in SD Card

Next Steps – Hardware Consideration I

- **Reposition ORB-SLAM2** as an offline evaluation tool or future extension unless the ROS environment can be established on another platform.
- While keeping the work technically professional while preserving the Monocular Visual Simultaneous Localization and Mapping ambition, and highlighting the contribution: a modular mobile robot autonomy stack that shows exactly where monocular SLAM integration becomes difficult on constrained, legacy embedded software environments like NVIDIA Jetson Nano.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Delighter

Delighter – Independent Cross-Platform Validation I

- While the NVIDIA JetPack 4.3 integration hit the blockers on the previous slide, the **same monocular ORB-SLAM3 pipeline** was independently built, debugged and validated end-to-end on **two different platforms outside the target hardware**: a laptop (WSL2/Ubuntu) and a smartphone camera stream.
- **Live tracking confirmed:** real-time keyframe tracking, local mapping and loop-closing threads visible in the Pangolin viewer.
- **Why this matters:** it isolates the JetBot shortfall to the **JetPack 4.3 / ROS Melodic integration layer** (Python/Boost bindings, `rospack`/`catkin` tooling) and not to the monocular VSLAM method, the calibration approach, or the underlying understanding of the algorithm.

Delighter – Engineering Behind the Validation I

Non-trivial fixes required to reach a stable, reproducible pipeline:

- **Build isolation:** removed leftover ROS/catkin `find_package` block from `CMakeLists.txt` to stop the linker silently merging two incompatible OpenCV versions (system 4.2 vs. source-built 4.4) into `libORB_SLAM3.so`.
- **Linking fix:** explicitly linked GLEW against Pangolin's OpenGL calls to resolve an undefined-reference failure at build time.
- **Streaming robustness:** added frame-validity checks and monotonic-timestamp enforcement in the capture loop to handle corrupted MJPEG frames and clock drift over a live network stream.
- **Camera calibration:** performed a full checkerboard calibration (intrinsics + distortion) rather than relying on a zero-distortion placeholder, since drift and tracking loss are highly sensitive to calibration accuracy.

Delighter – Engineering Behind the Validation II

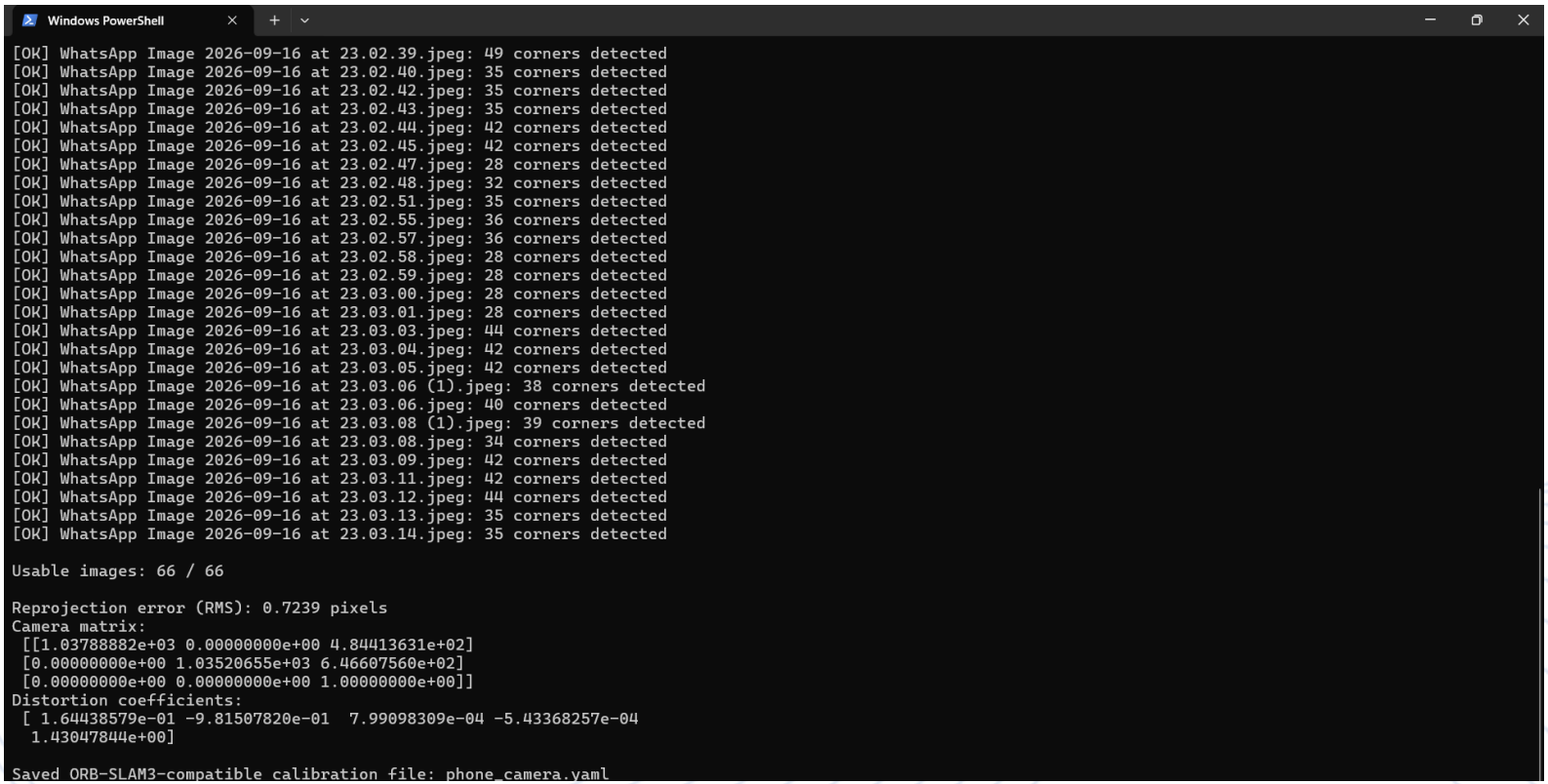


Abbildung: Mobile Camera Calibration

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

Vielen Dank

für Ihre Aufmerksamkeit